

03
PHASE

SQL & Databases

Query Data. Manage Databases.
Drive Insights.



Design
Databases



Write
Queries



Filter &
Aggregate



Analyze
Data



Ensure Data
Integrity



Structured Data. Smarter Decisions.

Unit 1

The Core Foundation: SELECT, WHERE, ORDER BY

What's Inside

- 1.1 Basic SELECT Syntax: Columns & Aliases
- 1.2 SELECT DISTINCT to Remove Duplicates
- 1.3 Filtering Rows with the WHERE Clause
- 1.4 Comparison Operators: =, <>, >, <, >=, <=
- 1.5 Logical Operators: AND, OR, NOT
- 1.6 BETWEEN, IN, and NOT IN Filters
- 1.7 Wildcard Filtering with LIKE (% , _)
- 1.8 Sorting Results with ORDER BY (ASC / DESC)
- 1.9 Limiting Results: LIMIT, TOP, FETCH NEXT
- 1.10 Column Aliasing with AS



Learnlytics
— .handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

UNIT 1 — THE CORE FOUNDATION

This unit covers the essential building blocks of SQL that every data analyst must master. These commands form the backbone of 80% of daily analytical queries. Master these, and you'll be able to retrieve, filter, sort, and shape data with confidence.

1.1 Basic SELECT Syntax: Columns & Aliases

DEFINITION

The SELECT statement is the most fundamental SQL command. It is used to retrieve data from one or more columns in a database table. You specify which columns you want to see, and SQL returns only those columns from the table. An alias is a temporary, custom name you give to a column or table in your query result — it does not change the actual database, just how the output looks.

SUB-TOPICS

SELECT *: Retrieves ALL columns from a table. Handy for quick exploration but avoid in production for performance reasons.

SELECT col1, col2: Retrieves only the specified columns. This is the recommended approach.

Column Alias (AS): Renames a column in the result set using the AS keyword. Makes output readable, especially for calculated fields.

Table Alias: A short nickname for a table, used mainly when working with multiple tables (covered in JOINS).

WHY IT MATTERS FOR DATA ANALYSTS

SELECT is the starting point of every SQL query. A data analyst uses it dozens of times a day to pull sales figures, customer records, product data, and more. Using aliases makes reports and dashboards more professional — stakeholders see 'Total Revenue' instead of 'SUM(ord_amt * qty)'.

REAL-WORLD ANALOGY

Think of it this way: Imagine a massive filing cabinet with hundreds of folders. SELECT lets you say 'Give me only the Name and Phone Number folders — nothing else.' Using an alias is like putting a sticky note on a folder that says 'Call This' so it's easier to reference later.

SYNTAX & EXAMPLE

Syntax	Example
-- Select specific columns SELECT column1, column2	-- Get employee name and salary SELECT first_name AS 'First Name',

```
FROM table_name;
```

```
-- Select with aliases  
SELECT column1 AS alias1,  
       column2 AS alias2  
FROM table_name;
```

```
-- Select all columns  
SELECT * FROM table_name;
```

```
last_name AS 'Last Name',  
salary AS 'Annual Salary'  
FROM employees;
```

```
-- Output columns will be labeled:  
-- First Name | Last Name | Annual  
Salary
```

INDUSTRY USE CASE

E-Commerce: An analyst at an online retailer writes `SELECT product_name AS 'Product', unit_price AS 'Price (USD)', stock_qty AS 'Available Stock' FROM products;` to generate a clean inventory report for the merchandising team. The aliases ensure the exported Excel sheet has human-readable column headers.

1.2 SELECT DISTINCT to Remove Duplicates

DEFINITION

`SELECT DISTINCT` is used to return only unique (non-repeating) values from a column or combination of columns. When you query a table, the same value may appear in multiple rows. `DISTINCT` filters out these repetitions and shows each unique value only once.

SUB-TOPICS

Single Column DISTINCT: Returns unique values from one column, e.g., all distinct cities where customers live.

Multi-Column DISTINCT: Returns unique combinations of multiple columns. Two rows are considered duplicates only if ALL selected columns match.

DISTINCT vs GROUP BY: Both can eliminate duplicates, but `GROUP BY` is used when you also want to aggregate (count, sum, etc.), while `DISTINCT` is purely for unique value retrieval.

WHY IT MATTERS FOR DATA ANALYSTS

Data analysts often need to understand the range or variety in a dataset before diving deep. How many unique products do we sell? In which cities do we have customers? `DISTINCT` answers these questions instantly without manual de-duplication.

REAL-WORLD ANALOGY

Think of it this way: You have a guest list where some names appear multiple times (a person RSVP'd twice). `SELECT DISTINCT` is like going through the list and writing each name only once — giving you a clean, unique guest list.

SYNTAX & EXAMPLE

Syntax	Example
<pre>-- Unique values from one column SELECT DISTINCT column1 FROM table_name; -- Unique combinations of two columns SELECT DISTINCT column1, column2 FROM table_name;</pre>	<pre>-- Find all unique countries of customers SELECT DISTINCT country FROM customers; -- Unique department + job title combos SELECT DISTINCT department, job_title FROM employees;</pre>

INDUSTRY USE CASE

Healthcare: A data analyst at a hospital uses `SELECT DISTINCT patient_id FROM appointments WHERE appointment_date = '2024-01-15';` to count how many unique patients visited on a given day — even if a single patient had two appointments that day, they're counted only once.

1.3 Filtering Rows with the WHERE Clause

DEFINITION

The WHERE clause is used to filter rows in a query. Instead of returning all rows from a table, WHERE lets you specify a condition — only rows that satisfy that condition are included in the result. WHERE is applied before any grouping or aggregation, so it filters raw data.

SUB-TOPICS

Basic condition: A simple test like `WHERE age = 30` returns only rows where the age column equals 30.

Text filtering: Use single quotes for text: `WHERE city = 'Mumbai'`. SQL string comparisons are case-insensitive in many databases.

Date filtering: Dates are also quoted: `WHERE order_date = '2024-06-01'`. Format may vary by database.

WHERE vs HAVING: WHERE filters individual rows before grouping; HAVING filters groups after a GROUP BY (covered later).

WHY IT MATTERS FOR DATA ANALYSTS

Without WHERE, every query would return the entire table — sometimes millions of rows. WHERE is the core filtering tool that lets analysts zoom into exactly the data slice they need: 'Show me only sales from Q4' or 'Give me customers in Delhi.'

REAL-WORLD ANALOGY

Think of it this way: Think of a WHERE clause like a bouncer at a club. The bouncer only lets in people who meet certain criteria (age 18+, on the guest list). Similarly, WHERE only 'lets in' rows that pass your condition.

SYNTAX & EXAMPLE

Syntax	Example
<pre>SELECT column1, column2 FROM table_name WHERE condition; -- WHERE with text WHERE city = 'Mumbai' -- WHERE with number WHERE salary > 50000 -- WHERE with date WHERE order_date = '2024-01-01'</pre>	<pre>-- Get all products priced above 1000 SELECT product_name, price FROM products WHERE price > 1000; -- Get orders from a specific city SELECT order_id, customer_name FROM orders WHERE city = 'Pune';</pre>

INDUSTRY USE CASE

Retail Banking: An analyst filters transaction data using WHERE transaction_type = 'WITHDRAWAL' AND amount > 50000; to flag large cash withdrawals for compliance review. Without WHERE, the query would return millions of transactions, making the analysis impossible.

1.4 Comparison Operators: =, <>, >, <, >=, <=

DEFINITION

Comparison operators are used inside the WHERE clause to compare a column's value against a specific value. The result of a comparison is always TRUE or FALSE — rows where the comparison is TRUE are included in the results.

SUB-TOPICS

= (Equal to): Returns rows where the column value exactly matches the given value. E.g., WHERE status = 'Active'.

<> or != (Not equal to): Returns rows where the value does NOT match. Both <> and != work in most databases.

> (Greater than): Returns rows where the value is strictly greater. E.g., WHERE age > 25.

< (Less than): Returns rows where the value is strictly less. E.g., WHERE price < 500.

>= (Greater than or equal to): Includes the boundary value. E.g., WHERE score >= 90 includes score = 90.

<= (Less than or equal to): Includes the boundary value. E.g., WHERE discount <= 10 includes discount = 10.

WHY IT MATTERS FOR DATA ANALYSTS

These are the building blocks of all data filtering. Whether you're finding high-value customers (salary >= 100000), excluding test accounts (status <> 'Test'), or finding discounted products (discount > 0), you'll use these operators in virtually every analytical query.

REAL-WORLD ANALOGY

Think of it this way: These work exactly like comparisons in everyday math. If your rule is 'Students who scored more than 60 pass', you write score > 60. SQL uses the same logic you learned in school.

SYNTAX & EXAMPLE

Syntax	Example
WHERE column = value -- Equal	-- Products priced at exactly 499
WHERE column <> value -- Not equal	SELECT * FROM products WHERE price =
WHERE column != value -- Not equal	499;
(alternate)	
WHERE column > value -- Greater than	-- Exclude cancelled orders
WHERE column < value -- Less than	SELECT * FROM orders WHERE status <>
WHERE column >= value -- Greater or	'Cancelled';
equal	
WHERE column <= value -- Less or	-- Students who passed (marks >= 50)
equal	SELECT name FROM students WHERE
	marks >= 50;
	-- Items with low stock (qty < 10)
	SELECT product FROM inventory WHERE
	qty < 10;

INDUSTRY USE CASE

Telecom Industry: A data analyst uses `WHERE churn_score >= 0.7` to identify customers at high risk of leaving the service. The team then targets these customers with retention offers. Without the `>=` operator, identifying this threshold group precisely would be impossible.

1.5 Logical Operators: AND, OR, NOT

DEFINITION

Logical operators are used to combine multiple conditions in a `WHERE` clause. **AND** requires ALL conditions to be true, **OR** requires AT LEAST ONE condition to be true, and **NOT** reverses a condition (true becomes false and vice versa). They allow you to build complex, precise filters.

SUB-TOPICS

AND: Both conditions must be TRUE for a row to be included. Narrows down results. E.g., `WHERE city = 'Delhi' AND age > 25`.

OR: At least one condition must be TRUE. Broadens results. E.g., `WHERE city = 'Delhi' OR city = 'Mumbai'`.

NOT: Reverses the condition. E.g., `WHERE NOT status = 'Inactive'` is equivalent to `WHERE status <> 'Inactive'`.

Operator Precedence: NOT is evaluated first, then AND, then OR. Always use parentheses () to make complex logic explicit and avoid bugs.

WHY IT MATTERS FOR DATA ANALYSTS

Real-world data questions always have multiple conditions. 'Show me high-value customers in the North region who haven't purchased in 90 days' requires AND. 'Show me sales from Mumbai or Pune' requires OR. These operators are essential for precise analysis.

REAL-WORLD ANALOGY

Think of it this way: AND is like saying 'I want coffee AND milk' — you need both. OR is like saying 'I'll have tea OR coffee' — either works. NOT is like saying 'Give me anything that is NOT decaf.'

SYNTAX & EXAMPLE

Syntax	Example
<pre>-- AND: both conditions must be true WHERE condition1 AND condition2 -- OR: at least one must be true WHERE condition1 OR condition2</pre>	<pre>-- High salary employees in IT dept SELECT name FROM employees WHERE department = 'IT' AND salary > 80000; -- Customers from two cities</pre>


```
-- NOT: reverses the condition
WHERE NOT condition
```

```
-- Combined (use parentheses!)
WHERE (condition1 OR condition2)
AND condition3
```

```
SELECT name FROM customers
WHERE city = 'Delhi'
OR city = 'Chennai';
```

```
-- All orders except pending
SELECT * FROM orders
WHERE NOT status = 'Pending';
```

INDUSTRY USE CASE

Insurance: An actuary uses WHERE age >= 18 AND age <= 35 AND policy_type = 'Life' to segment young adult life insurance customers for a targeted pricing analysis. The AND operator ensures only exact matches across all three criteria are returned.

1.6 BETWEEN, IN, and NOT IN Filters

DEFINITION

These are shorthand filtering operators that make queries more readable and concise. BETWEEN filters rows within a range of values (inclusive on both ends). IN filters rows where a column matches any value in a list. NOT IN excludes rows that match any value in a list.

SUB-TOPICS

BETWEEN ... AND ...: Equivalent to value >= lower AND value <= upper. Works with numbers, dates, and text. Both boundary values are included.

NOT BETWEEN: The inverse — returns rows where the value is outside the specified range.

IN (...): Equivalent to multiple OR conditions. E.g., WHERE city IN ('Delhi', 'Mumbai', 'Pune') is cleaner than three OR clauses.

NOT IN (...): Excludes all rows that match any value in the list. Be careful with NULLs — NOT IN can behave unexpectedly if the list contains NULL values.

WHY IT MATTERS FOR DATA ANALYSTS

These operators dramatically simplify query writing. Instead of writing WHERE age >= 18 AND age <= 35, you write WHERE age BETWEEN 18 AND 35. For IN, replacing 10 OR conditions with one IN list is cleaner, faster to write, and easier to maintain.

REAL-WORLD ANALOGY

Think of it this way: BETWEEN is like saying 'Give me all flights between 9 AM and 5 PM.' IN is like a VIP list at a party — only people on the list get in. NOT IN is the blacklist — everyone on it is denied entry.

SYNTAX & EXAMPLE

Syntax	Example
<pre>-- BETWEEN (inclusive range) WHERE column BETWEEN value1 AND value2 -- NOT BETWEEN WHERE column NOT BETWEEN value1 AND value2 -- IN (match any in list) WHERE column IN (val1, val2, val3) -- NOT IN (exclude list) WHERE column NOT IN (val1, val2, val3)</pre>	<pre>-- Orders in Q1 2024 SELECT * FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-03- 31'; -- Top-tier products (price range) SELECT * FROM products WHERE price BETWEEN 1000 AND 5000; -- Sales from 3 specific regions SELECT * FROM sales WHERE region IN ('North', 'South', 'East');</pre>

INDUSTRY USE CASE

Finance: A portfolio analyst uses `WHERE trade_date BETWEEN '2024-01-01' AND '2024-12-31' AND ticker IN ('AAPL', 'MSFT', 'GOOG')` to pull all annual trades for a specific set of stocks for year-end reporting.

1.7 Wildcard Filtering with LIKE (% , _)

DEFINITION

The LIKE operator is used to filter text data based on a pattern rather than an exact match. It uses two special wildcard characters: % (percent) represents zero or more characters of any kind, and _ (underscore) represents exactly one character of any kind. LIKE is case-insensitive in most databases.

SUB-TOPICS

% Wildcard: Matches any sequence of characters (including none). 'A%' matches 'Apple', 'Amazon', 'A'. '%co%' matches anything containing 'co'.

_ Wildcard: Matches exactly ONE character. 'J_hn' matches 'John' or 'Jahn' but NOT 'Jon' or 'Johann'.

NOT LIKE: Returns rows that do NOT match the pattern.

ILIKE (PostgreSQL): Case-insensitive version of LIKE. Standard LIKE may be case-sensitive in some databases like PostgreSQL.

Escaping wildcards: To search for a literal % or _, use an escape character: WHERE code LIKE '10!%' ESCAPE '!'.

WHY IT MATTERS FOR DATA ANALYSTS

Real-world data is messy and inconsistent. Customer emails, product codes, and address fields often need pattern-based searching. LIKE enables fuzzy matching that exact operators cannot do — essential for data cleaning, search features, and exploratory analysis.

REAL-WORLD ANALOGY

Think of it this way: LIKE is like using a search bar on Google. If you type 'iPhone %' it finds 'iPhone 13', 'iPhone 14 Pro', 'iPhone SE' etc. The % is like a wildcard in a search — fill in any value here. The _ is more specific — exactly one character, like a fill-in-the-blank with one letter.

SYNTAX & EXAMPLE

Syntax	Example
<pre>-- Pattern matching with LIKE WHERE column LIKE 'pattern' -- % = any number of characters WHERE name LIKE 'A%' -- starts with A WHERE email LIKE '%@gmail.com' -- ends with WHERE city LIKE '%ban%' -- contains 'ban' -- _ = exactly one character WHERE code LIKE 'A_C' -- A, any char, C</pre>	<pre>-- Customers whose name starts with 'S' SELECT * FROM customers WHERE name LIKE 'S%'; -- Products containing 'Phone' SELECT * FROM products WHERE product_name LIKE '%Phone%'; -- Email IDs from Gmail only SELECT email FROM users WHERE email LIKE '%@gmail.com';</pre>

INDUSTRY USE CASE

HR Management: An HR analyst uses WHERE employee_id LIKE 'EMP_2024%' to pull all employees who joined in 2024, where the ID format is 'EMP_2024_001', 'EMP_2024_002', etc. This pattern-based filter avoids manually listing hundreds of IDs.

1.8 Sorting Results with ORDER BY (ASC / DESC)

DEFINITION

The ORDER BY clause is used to sort the rows in your query result. By default, rows come back in no particular guaranteed order — the database returns them however it finds them. ORDER BY lets you control the sort: ASC (ascending, A→Z, 1→10) is the default, and DESC (descending, Z→A, 10→1) reverses it. You can sort by multiple columns.

SUB-TOPICS

ASC (Ascending): Sorts from lowest to highest (numbers) or A to Z (text). This is the default — you don't need to write ASC explicitly, but it's good practice.

DESC (Descending): Sorts from highest to lowest (numbers) or Z to A (text). Used for 'Top N' type queries like 'most expensive products'.

Multi-column sorting: ORDER BY col1 ASC, col2 DESC — first sorts by col1, then for ties in col1, sorts by col2.

Sorting by column position: ORDER BY 2 sorts by the second column in the SELECT list. Less readable but sometimes convenient.

NULL handling: NULLs are typically sorted last in ASC and first in DESC. Behavior varies by database.

WHY IT MATTERS FOR DATA ANALYSTS

Sorted data is essential for meaningful reports and dashboards. 'Top 10 sales reps', 'Most recent orders', 'Lowest-rated products' — all require ORDER BY. Without it, report data appears random, making patterns invisible.

REAL-WORLD ANALOGY

Think of it this way: ORDER BY is like a leaderboard. ASC puts the beginner at the top (scores 10, 20, 30...). DESC puts the champion at the top (scores 100, 95, 88...). Multi-column ORDER BY is like sorting students first by class, then by rank within each class.

SYNTAX & EXAMPLE

Syntax	Example
<pre>SELECT column1, column2 FROM table_name ORDER BY column1 ASC; -- default -- Descending sort ORDER BY column1 DESC; -- Multi-column sort ORDER BY column1 ASC, column2 DESC;</pre>	<pre>-- Top selling products (highest first) SELECT product_name, total_sales FROM products ORDER BY total_sales DESC; -- Employees sorted by dept, then salary SELECT name, department, salary FROM employees ORDER BY department ASC, salary DESC;</pre>

INDUSTRY USE CASE

Logistics: A supply chain analyst uses ORDER BY delivery_date ASC to prioritize shipments — the earliest due deliveries appear at the top of the list, helping the operations team focus on urgent dispatches first.

1.9 Limiting Results: LIMIT, TOP, FETCH NEXT

DEFINITION

These clauses restrict how many rows are returned by a query. Instead of fetching all matching rows (which could be millions), you tell the database to return only the first N rows. The syntax varies by database: MySQL/PostgreSQL use LIMIT, SQL Server uses TOP, and the SQL standard uses FETCH NEXT ... ROWS ONLY. OFFSET allows you to skip a certain number of rows (used for pagination).

SUB-TOPICS

LIMIT n (MySQL, PostgreSQL): Returns only the first n rows. E.g., LIMIT 10 returns 10 rows.

TOP n (SQL Server): Written at the start of SELECT: SELECT TOP 10 * FROM table. Returns the top n rows.

FETCH NEXT n ROWS ONLY (Standard SQL): ANSI SQL standard. Used with OFFSET: OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY.

OFFSET: Skips a specified number of rows before returning results. Essential for pagination (page 1 = OFFSET 0, page 2 = OFFSET 10, etc.).

LIMIT with ORDER BY: Always combine LIMIT with ORDER BY. Without ORDER BY, the 'first' N rows are random. LIMIT + ORDER BY gives meaningful results like 'Top 5 customers by revenue'.

WHY IT MATTERS FOR DATA ANALYSTS

Returning all rows from a large table is slow and wastes resources. Analysts use LIMIT to quickly preview data, test queries on a small sample, or build paginated reports. It's also crucial for performance optimization in production dashboards.

REAL-WORLD ANALOGY

Think of it this way: Imagine Google showing you 5 billion search results at once — useless! Instead, it shows 10 per page. LIMIT is the same concept: give me just the first N results, not everything in the database. OFFSET is clicking 'next page.'

SYNTAX & EXAMPLE

Syntax	Example
<pre>-- MySQL / PostgreSQL SELECT * FROM table LIMIT 10; -- With OFFSET (skip 20, get next 10) SELECT * FROM table LIMIT 10 OFFSET 20; -- SQL Server SELECT TOP 10 * FROM table; -- ANSI SQL Standard SELECT * FROM table OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY;</pre>	<pre>-- Top 5 most expensive products SELECT product_name, price FROM products ORDER BY price DESC LIMIT 5; -- Page 2 of results (rows 11-20) SELECT * FROM customers ORDER BY customer_id LIMIT 10 OFFSET 10;</pre>

INDUSTRY USE CASE

Media Streaming: A data analyst at a streaming platform uses `SELECT title, view_count FROM shows ORDER BY view_count DESC LIMIT 10;` to generate the 'Top 10 Most Watched Shows This Week' list for the homepage. `LIMIT` ensures the query is fast and returns exactly the data the frontend needs.

1.10 Column Aliasing with AS

DEFINITION

An alias is a temporary custom name given to a column or table in a query's output. You use the `AS` keyword to assign an alias. Aliases only exist for the duration of the query — they do not modify the actual column name in the database. Aliases are especially useful for renaming calculated columns, making output more readable, and simplifying complex expressions.

SUB-TOPICS

Column Alias: Renames a column in the result: `SELECT salary AS 'Annual Salary'`. The output column is labeled 'Annual Salary'.

Expression Alias: Essential for calculated columns which have no natural name: `SELECT price * quantity AS 'Total Revenue'`.

Optional AS keyword: In most databases, `AS` is optional — `SELECT salary 'Annual Salary'` works too. But using `AS` improves clarity.

Alias with spaces: Use single or double quotes (database-dependent) for aliases with spaces: `AS 'Total Sales'` or `AS "Total Sales"`.

Alias in ORDER BY: You can reference a column alias in ORDER BY: ORDER BY 'Total Revenue' DESC. You generally CANNOT use it in WHERE (use a subquery instead).

WHY IT MATTERS FOR DATA ANALYSTS

Data analysts create reports and dashboards consumed by non-technical stakeholders — managers, executives, clients. Raw column names like `cust_acq_dt` or `avg_ord_val_90d` are meaningless to them. Aliases produce clean, professional output: 'Customer Since' and 'Avg. Order Value (90 days)'. They are also essential when using aggregate functions.

REAL-WORLD ANALOGY

Think of it this way: An alias is like a nickname. Your name might be 'Shubhankar Raghavendra Krishnamurthy' in official records, but everyone calls you 'Shubh.' The alias 'Shubh' is easier to use in conversation — just like SQL aliases make column names easier to read in reports.

SYNTAX & EXAMPLE

Syntax	Example
<pre>-- Basic column alias SELECT column_name AS alias_name FROM table_name; -- Alias for calculated expression SELECT price * qty AS 'Revenue' FROM orders; -- Alias used in ORDER BY SELECT salary * 12 AS annual_pay FROM employees ORDER BY annual_pay DESC;</pre>	<pre>-- Clean sales report SELECT cust_id AS 'Customer ID', first_nm AS 'First Name', last_nm AS 'Last Name', ord_dt AS 'Order Date', price * qty AS 'Total Amount' FROM orders ORDER BY 'Total Amount' DESC;</pre>

INDUSTRY USE CASE

Digital Marketing: A marketing analyst builds a weekly performance report using aliases: `SELECT campaign_nm AS 'Campaign', impressions AS 'Total Views', clicks AS 'Clicks', (clicks/impressions)*100 AS 'CTR (%)' FROM campaigns;` The output goes directly into an executive PowerPoint — aliases ensure every column header is presentation-ready.

QUICK REFERENCE — UNIT 1 CHEAT SHEET

Command	Purpose	Quick Example	Key Tip
SELECT cols	Retrieve specific columns	<code>SELECT name, age FROM users;</code>	<i>Always specify columns, avoid <code>SELECT *</code></i>
DISTINCT	Remove duplicate rows	<code>SELECT DISTINCT city FROM orders;</code>	<i>Use for unique value lists</i>
WHERE	Filter rows by condition	<code>WHERE price > 100</code>	<i>Applied before grouping</i>
=, <>, >, <	Compare values	<code>WHERE age >= 18</code>	<i><> means 'not equal'</i>
AND / OR / NOT	Combine conditions	<code>WHERE a=1 AND b=2</code>	<i>Use parentheses for clarity</i>
BETWEEN	Range filter (inclusive)	<code>WHERE age BETWEEN 18 AND 35</code>	<i>Includes both boundary values</i>
IN / NOT IN	List match/exclusion	<code>WHERE city IN ('Delhi', 'Pune')</code>	<i>Cleaner than multiple ORs</i>
LIKE	Pattern matching	<code>WHERE name LIKE 'A%'</code>	<i>% = many chars, _ = one char</i>
ORDER BY	Sort results	<code>ORDER BY salary DESC</code>	<i>Always pair with <code>LIMIT</code> for Top-N</i>
LIMIT / TOP	Restrict row count	<code>LIMIT 10 / TOP 10</code>	<i>Use <code>OFFSET</code> for pagination</i>
AS (Alias)	Rename column in output	<code>SELECT price AS 'Cost'</code>	<i>Use for calculated columns</i>

Learnlytics

.handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

UNIT 1 — Interview Base Unit Tests

The Core Foundation: SELECT, WHERE, ORDER BY

SQL for Data Analysts | Learnlytics.handbook | 2026

LEVEL 1: Warmup Test — 10 Basic Questions

LEVEL 2: Interviewer Test — 10 Intermediate + Semi Advance Questions

Instructions

Note: Do not skip the struggle! Force yourself to write the code and explanations for the Descriptive, Scenario-Based, and Coding questions first. Once you have completed the test, check your work against the detailed solutions provided in the **SQL Interview Base Answer Sheet PDF**.

- Attempt all 20 questions across the three levels before checking answers.
- MCQ: Circle or write the letter of the best option.
- Descriptive / Scenario questions: Write your answer in the provided box.
- Coding questions: Write your SQL query in the code box (any SQL dialect).
- Topics: SELECT, DISTINCT, WHERE, Comparison & Logical Operators, BETWEEN, IN, LIKE, ORDER BY, LIMIT/TOP, Aliases.

Total: 20 Questions | 2 Levels | Unit 1 (Topics 1.1 – 1.10)

Learnlytics
handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Level 1: Warmup Test — 10 Basic Questions

MCQs | Descriptive | Scenario-Based | Coding | Database Coding

MCQ QUESTIONS (3 Questions)

Choose the single best answer.

Q1 MCQ — Topic: Basic SELECT Syntax

Which SQL keyword is used to retrieve all columns from a table called employees?

A. SELECT employees;	B. SELECT * FROM employees;
C. GET ALL FROM employees;	D. FETCH employees.*;

Q2 MCQ — Topic: WHERE Clause & Comparison Operators

An analyst wants all orders where order_amount is more than 5000. Which WHERE clause is correct?

A. WHERE order_amount = 5000	B. WHERE order_amount <> 5000
C. WHERE order_amount > 5000	D. WHERE order_amount < 5000

Q3 MCQ — Topic: ORDER BY Default Sort

What is the default sort direction of ORDER BY in SQL when no ASC or DESC keyword is written?

A. Descending (Z to A, 9 to 1)	B. Random — database decides
C. Ascending (A to Z, 1 to 9)	D. Order of insertion into the table

DESCRIPTIVE QUESTION (1 Question)

Q4 Descriptive — Topic: Column Aliases (AS)

Explain the purpose of column aliases in SQL. Why should a data analyst use the AS keyword in SELECT statements? What is the difference between aliasing a column in a query result and actually renaming the column in the database table?

Answer: Your Answer (Write below)

SCENARIO-BASED DESCRIPTIVE QUESTIONS (2 Questions)

Q5 Scenario-Based — Topic: WHERE + Logical Operators

Scenario: You are a data analyst at a retail bank. Your manager asks you to find all customers from Mumbai with a balance greater than Rs.1,00,000 whose account status is NOT 'Suspended'.

Describe how you would write the WHERE clause. Which logical operators would you use, and why is each necessary?

Answer: Your Answer (Write below)

Q6 Scenario-Based — Topic: SELECT DISTINCT

Scenario: You work at an e-commerce company with an orders table (order_id, customer_id, product_category, city, order_amount). Your manager asks: 'How many unique cities are our customers ordering from?'

Why is SELECT DISTINCT city FROM orders the right approach here? What would happen with SELECT city FROM orders? How does DISTINCT differ from GROUP BY for this question?

Answer: Your Answer (Write below)

CODING QUESTIONS — SQL (2 Questions)

Q7 Coding — Topic: *SELECT + Aliases + ORDER BY*

Write a SQL query on the employees table to retrieve first_name (alias: 'First Name'), last_name (alias: 'Last Name'), and salary (alias: 'Annual Salary'). Sort by salary from highest to lowest.

SQL Query: Write your SQL query here *(Write your SQL below)*

Q8 Coding — Topic: *WHERE + BETWEEN + IN*

Write a SQL query on the products table (product_id, product_name, category, price, stock_qty) to find all products priced between 500 and 2000 (inclusive) AND in category 'Electronics', 'Mobile', or 'Accessories'. Show product_name, category, price only.

SQL Query: Write your SQL query here *(Write your SQL below)*

LEARN SMART. ANALYZE BETTER. GROW FASTER.

DATABASE SCENARIO-BASED CODING (2 Questions)

Q9 Database Coding — Topic: *WHERE + LIKE + LIMIT*

Database: A hospital table patients has columns: patient_id, full_name, city, doctor_name, admission_date, discharge_date.

Task: Find the 5 most recently admitted patients whose doctor's name starts with 'Dr. S'. Show patient_id, full_name, doctor_name, admission_date. Sort newest admission first.

SQL Query: Write your SQL query here *(Write your SQL below)*

Q10 Database Coding — Topic: SELECT DISTINCT + WHERE + ORDER BY

Database: A logistics table shipments has columns: shipment_id, origin_city, destination_city, carrier, status, shipment_date.

Task: Get a unique list of all destination cities where at least one shipment has status 'Delayed'. Sort cities alphabetically.

SQL Query: Write your SQL query here *(Write your SQL below)*

Level 2: Interviewer Test — 10 Intermediate Questions

MCQs | Descriptive | Scenario-Based | Coding | Database Coding

MCQ QUESTIONS (2 Questions)

Choose the single best answer.

Q11 MCQ — Topic: LIKE Wildcards

An analyst writes: WHERE product_code LIKE 'INV_2024%'. Which of the following product_code values would this query INCLUDE in the results?

A. INVX2024001	B. INV-2024-102
C. INV_2024_305	D. 2024_INV_001

Q12 MCQ — Topic: AND/OR Operator Precedence

What does this WHERE clause return? WHERE city = 'Delhi' OR city = 'Mumbai' AND salary > 80000

A. All Delhi rows, plus Mumbai rows where salary > 80000

B. Only rows where city is Delhi or Mumbai AND salary > 80000

C. Only Delhi rows where salary > 80000

D. All rows from Delhi and all rows from Mumbai regardless of salary

DESCRIPTIVE QUESTIONS (2 Questions)

Q13 Descriptive — Topic: BETWEEN vs >= AND <=

A junior analyst writes: WHERE order_date >= '2024-01-01' AND order_date <= '2024-03-31'. A senior suggests using BETWEEN instead. Explain: (a) Are the results identical? (b) What are BETWEEN's boundary value rules? (c) When would you prefer one syntax over the other?

Answer: Your Answer (Write below)

Q14 Descriptive — Topic: IN vs OR + NOT IN Pitfall

Compare these two queries — are results identical? Which is preferred in production SQL and why? Query A: WHERE dept = 'HR' OR dept = 'Finance' OR dept = 'Legal'. Query B: WHERE dept IN ('HR','Finance','Legal'). Also describe one critical pitfall when using NOT IN with a subquery.

Answer: Your Answer (Write below)

SCENARIO-BASED DESCRIPTIVE QUESTIONS (2 Questions)

Q15 Scenario-Based — Topic: LIKE Pattern Matching for Data Audit

Scenario: You are a data quality analyst at a telecom company. 2 million customer records were imported into raw_customers. You suspect emails have formatting errors — missing '@', extra spaces, wrong domains.

Describe how to use LIKE, %, and _ to: (a) Find records missing '@' in the email column. (b) Find emails ending with '@gmail.com'. (c) Explain the difference between '%gmail%' and '%@gmail.com' as filters.

Answer: Your Answer (Write below)

Q16 Scenario-Based — Topic: ORDER BY + LIMIT for Rankings

Scenario: You are a sales analyst at an FMCG company. Your VP wants 'Top 10 highest-revenue products this quarter' every Monday. The products table has: product_id, product_name, category, quarterly_revenue, units_sold.

Describe your approach: (a) Which clauses in which order? (b) How to handle ties in quarterly_revenue? (c) How to change the query to return 'Bottom 10 worst-performing products' instead?

Answer: Your Answer (Write below)

CODING QUESTIONS — SQL (2 Questions)

Q17 Coding — Topic: LIKE + BETWEEN + ORDER BY

Write a SQL query on the customers table (customer_id, full_name, email, city, registration_date) to find customers who: registered between 1 Jan 2023 and 31 Dec 2023 (inclusive), AND have an email ending with '@outlook.com'. Return full_name, email, city, registration_date. Sort by registration_date newest first.

SQL Query: Write your SQL query here *(Write your SQL below)*

Q18 Coding — Topic: NOT IN + ORDER BY + LIMIT + Alias

Write a query on orders (order_id, customer_id, product_name, status, order_amount, order_date) to find the 10 most expensive orders NOT in status 'Cancelled' or 'Returned'. Return order_id, product_name, order_amount aliased as 'Order Value', status. Sort by order_amount descending.

SQL Query: Write your SQL query here *(Write your SQL below)*

DATABASE SCENARIO-BASED CODING (2 Questions)

Q19 Database Coding — Topic: Multi-Condition WHERE + LIKE + ORDER BY

Database: HR table employees has: emp_id, full_name, department, designation, salary, hire_date, city.

Task: Find all employees with 'Analyst' in their designation, in the 'IT' or 'Data' department, earning more than Rs.70,000. Sort by salary highest first. Return emp_id, full_name, designation, department, salary.

SQL Query: Write your SQL query here (Write your SQL below)

Q20 Database Coding — Topic: BETWEEN + IN + ORDER BY + LIMIT

Database: Pharma table drug_sales has: sale_id, drug_name, region, sale_date, units_sold, revenue, sales_rep.

Task: Find top 5 highest-revenue sales in Q1 2024 (Jan-Mar 2024) from regions 'North', 'South', or 'West'. Show sale_id, drug_name, region, sale_date, revenue aliased as 'Q1 Revenue'. Sort by revenue descending.

SQL Query: Write your SQL query here (Write your SQL below)

Note: Do not skip the struggle! Force yourself to write the code and explanations for the Descriptive, Scenario-Based, and Coding questions first. Once you have completed the test, check your work against the detailed solutions provided in the **Unit 1 Interview Base Answer Sheet PDF**.

Learnlytics.handbook | Unit 1 Interview Base Unit Tests | Copyright 2026 |

<https://learnlyticsnotes.vercel.app/>

UNIT 1 — Interview Base Unit Test Answers

The Core Foundation: SELECT, WHERE, ORDER BY
SQL for Data Analysts | Learnlytics.handbook | 2026

MCQ ANSWER KEY

Answers for Multiple Choice Questions Only | All Three Levels

Q1	Level 1	Topic: Basic SELECT Syntax	Correct Answer: B
----	---------	----------------------------	-------------------

Explanation: `SELECT * FROM employees;` is the correct ANSI SQL syntax. The asterisk (*) wildcard selects all columns. The other options use `GET` (not SQL), `FETCH` (not standard SELECT), or are missing `FROM`.

Q2	Level 1	Topic: WHERE + Comparison Operators	Correct Answer: C
----	---------	-------------------------------------	-------------------

Explanation: `WHERE order_amount > 5000` returns rows strictly greater than 5000. Option A uses `=` (exact match only), B uses `<>` (not equal), and D is contradictory – a value cannot be both `>= 5000` AND `< 5000`.

Q3	Level 1	Topic: ORDER BY Default Direction	Correct Answer: C
----	---------	-----------------------------------	-------------------

Explanation: SQL's `ORDER BY` defaults to `ASC` (Ascending – A to Z, 1 to 9) when no direction keyword is specified. You must explicitly write `DESC` to sort in descending order.

Q11	Level 2	Topic: LIKE Wildcards	Correct Answer: C
-----	---------	-----------------------	-------------------

Explanation: `'INV_2024_305'` matches `'INV_2024%'` – the `_` in the pattern matches the underscore character in the value (as a single-character wildcard), and `%` matches `'305'`. `'INVS2024001'` and `'INV-2024-102'` don't start with `'INV_2024'`. `'2024_INV_001'` doesn't start with `'INV'`.

Q12	Level 2	Topic: AND/OR Operator Precedence	Correct Answer: A
-----	---------	-----------------------------------	-------------------

Explanation: `AND` has higher precedence than `OR`. SQL evaluates: `city='Delhi'` OR `(city='Mumbai' AND salary>80000)`. Result: ALL Delhi rows PLUS only Mumbai rows where salary exceeds 80,000. Without parentheses the `AND` binds before `OR` – a classic precedence trap.

Level 1: Warmup Test

MCQs | Descriptive | Scenario-Based | Coding | Database Coding

DESCRIPTIVE QUESTION (1 Question)

Q4 Descriptive — Topic: Column Aliases (AS)

Explain the purpose of column aliases in SQL. Why should a data analyst use the AS keyword in SELECT statements? What is the difference between aliasing a column in a query result and actually renaming the column in the database table?

Answer: (Write below)

When I think about column aliases, I look at them as temporary nicknames for our data output. The main purpose of the AS keyword is to make the results more readable and intuitive for whoever is looking at the report.

Why use it? Often, we write queries with calculations or aggregations like `SUM(revenue) - SUM(cost)`. Without an alias, the column header in the result will literally be that whole ugly formula. By adding `AS net_profit`, the output instantly becomes clean and professional. The critical difference between aliasing and actually renaming a column is permanence. Aliasing only affects the display of that specific query's output—it's entirely superficial. Renaming a column using an `ALTER TABLE` statement fundamentally changes the database schema itself, which affects every future query and application connected to that table.

```
-- Aliasing just for this output
SELECT first_name, (salary * 0.10) AS bonus
FROM employees;
```

handbook

SCENARIO-BASED DESCRIPTIVE QUESTIONS (2 Questions)

Q5 Scenario-Based — Topic: WHERE + Logical Operators

Scenario: You are a data analyst at a retail bank. Your manager asks you to find all customers from Mumbai with a balance greater than Rs.1,00,000 whose account status is NOT 'Suspended'.

Describe how you would write the WHERE clause. Which logical operators would you use, and why is each necessary?

Answer: (Write below)

For this scenario, I need to filter the data down using three specific conditions. To tie them all together, I would use the AND operator because the manager wants customers who meet *all* these criteria simultaneously.

Here is how I would write the WHERE clause:

```
WHERE city = 'Mumbai'
AND balance > 100000
AND status != 'Suspended'
```

Q6 Scenario-Based — Topic: SELECT DISTINCT

Scenario: You work at an e-commerce company with an orders table (order_id, customer_id, product_category, city, order_amount). Your manager asks: 'How many unique cities are our customers ordering from?'

Why is SELECT DISTINCT city FROM orders the right approach here? What would happen with SELECT city FROM orders? How does DISTINCT differ from GROUP BY for this question?

Answer: (Write below)

If the manager wants to know the *unique* cities, SELECT DISTINCT city FROM orders is the perfect approach because it explicitly filters out duplicate values in the result set.

If I just ran SELECT city FROM orders, the database would return a row for every single order. If 5,000 orders came from Bangalore, 'Bangalore' would print 5,000 times, which doesn't answer the manager's question.

While GROUP BY city would technically yield the exact same visual list of unique cities, DISTINCT is semantically cleaner when we aren't doing any math. I prefer to save GROUP BY for when I need to perform aggregations, like counting *how many* orders came from each city.

handbook

CODING QUESTIONS — SQL (2 Questions)

Q7 Coding — Topic: SELECT + Aliases + ORDER BY

Write a SQL query on the employees table to retrieve first_name (alias: 'First Name'), last_name (alias: 'Last Name'), and salary (alias: 'Annual Salary'). Sort by salary from highest to lowest.

SQL Query: Write your SQL query here (Write your SQL below)

For this one, I just need to map the aliases properly and ensure the sort goes from highest to lowest using DESC.

```
SELECT
  first_name AS 'First Name',
  last_name AS 'Last Name',
  salary AS 'Annual Salary'
FROM
  employees
ORDER BY
  salary DESC;
```

Q8 Coding — Topic: WHERE + BETWEEN + IN

Write a SQL query on the products table (product_id, product_name, category, price, stock_qty) to find all products priced between 500 and 2000 (inclusive) AND in category 'Electronics', 'Mobile', or 'Accessories'. Show product_name, category, price only.

SQL Query: Write your SQL query here (Write your SQL below)

This is a great place to use BETWEEN and IN because they make the query much more readable than stringing together a bunch of > = and OR statements.

```
SELECT
  product_name,
  category,
  price
FROM
  products
WHERE
  price BETWEEN 500 AND 2000
  AND category IN ('Electronics', 'Mobile', 'Accessories');
```

handbook

DATABASE SCENARIO-BASED CODING (2 Questions)

Q9 Database Coding — Topic: WHERE + LIKE + LIMIT

Database: A hospital table patients has columns: patient_id, full_name, city, doctor_name, admission_date, discharge_date.

Task: Find the 5 most recently admitted patients whose doctor's name starts with 'Dr. S'. Show patient_id, full_name, doctor_name, admission_date. Sort newest admission first.

SQL Query: Write your SQL query here (Write your SQL below)

To find doctors starting with 'Dr. S', I'll use the LIKE operator with the % wildcard. To get the most recent admissions, sorting the date descending and limiting to 5 handles the logic perfectly.

```
SELECT
  patient_id,
  full_name,
  doctor_name,
  admission_date
FROM
  patients
WHERE
  doctor_name LIKE 'Dr. S%'
ORDER BY
  admission_date DESC
LIMIT 5;
```

Q10 Database Coding — Topic: SELECT DISTINCT + WHERE + ORDER BY

Database: A logistics table shipments has columns: shipment_id, origin_city, destination_city, carrier, status, shipment_date.

Task: Get a unique list of all destination cities where at least one shipment has status 'Delayed'. Sort cities alphabetically.

SQL Query: Write your SQL query here (Write your SQL below)

Here, we need unique destinations, but only for delayed shipments. I'll filter the delayed ones first, then apply DISTINCT to the destination city, and finally sort them.

```
SELECT DISTINCT
  destination_city
FROM
  shipments
WHERE
  status = 'Delayed'
ORDER BY
  destination_city ASC;
```

Level 2: Interviewer Test

MCQs | Descriptive | Scenario-Based | Coding | Database Coding

DESCRIPTIVE QUESTIONS (2 Questions)

Q13 Descriptive — Topic: BETWEEN vs >= AND <=

A junior analyst writes: `WHERE order_date >= '2024-01-01' AND order_date <= '2024-03-31'`. A senior suggests using `BETWEEN` instead. Explain: (a) Are the results identical? (b) What are `BETWEEN`'s boundary value rules? (c) When would you prefer one syntax over the other?

Answer: *(Write below)*

When I first started writing SQL, I used to think these were just two ways of typing the exact same thing. Functionally, they do achieve the same goal, but there are a few nuances to keep in mind.

(a) Are the results identical? Yes, assuming the date logic is exact. `date >= '2024-01-01' AND date <= '2024-03-31'` will return the exact same rows as using `BETWEEN`.

(b) What are `BETWEEN`'s boundary value rules? The most important thing to remember about `BETWEEN` is that it is **inclusive**. It includes both the start value ('2024-01-01') and the end value ('2024-03-31').

(c) When would you prefer one syntax over the other? I strongly prefer `BETWEEN` for standard date ranges and numbers because it is much cleaner and easier to read. However, if my dates have timestamp data attached (like `2024-03-31 14:30:00`), `BETWEEN '2024-01-01' AND '2024-03-31'` will actually miss anything on March 31st that happened after midnight. In those cases, using `>= '2024-01-01' AND < '2024-04-01'` is the safer, more precise choice.

Q14 Descriptive — Topic: `IN` vs `OR` + `NOT IN` Pitfall

Compare these two queries — are results identical? Which is preferred in production SQL and why? Query A: `WHERE dept = 'HR' OR dept = 'Finance' OR dept = 'Legal'`. Query B: `WHERE dept IN ('HR','Finance','Legal')`. Also describe one critical pitfall when using `NOT IN` with a subquery.

Answer: *(Write below)*

(a) Are the results identical? Which is preferred? The results of Query A and Query B are completely identical. However, in production SQL, Query B (using `IN`) is heavily preferred. From a practical standpoint, it is much easier to read, faster to type, and less prone to copy-paste errors. Plus, many database engines are optimized to evaluate an `IN` list faster than evaluating a long chain of separate `OR` conditions.

(b) The `NOT IN` Subquery Pitfall This is a trap I've definitely fallen into! The critical pitfall with `NOT IN` happens when you use it against a subquery that happens to return a `NULL` value. Because SQL evaluates `NULL` as "unknown", checking if a value is "not in" a list containing an unknown results in an unknown. The practical result? Your query will suddenly return **zero rows**, even if there are legitimate non-matching records. To avoid this, it's usually safer to use `NOT EXISTS`.

SCENARIO-BASED DESCRIPTIVE QUESTIONS (2 Questions)

Q15 Scenario-Based — Topic: LIKE Pattern Matching for Data Audit

Scenario: You are a data quality analyst at a telecom company. 2 million customer records were imported into raw_customers. You suspect emails have formatting errors — missing '@', extra spaces, wrong domains.

Describe how to use LIKE, %, and _ to: (a) Find records missing '@' in the email column. (b) Find emails ending with '@gmail.com'. (c) Explain the difference between '%gmail%' and '%@gmail.com' as filters.

Answer: (Write below)

As a data quality analyst, standardizing raw inputs is a huge part of the job. The LIKE operator is my go-to tool for finding these formatting anomalies.

(a) Finding records missing '@': I would look for any string that does not contain the symbol anywhere inside it. `WHERE email NOT LIKE '%@%'`

(b) Finding emails ending with '@gmail.com': I would use the % wildcard at the very beginning to say "I don't care what comes before this, as long as it ends exactly this way." `WHERE email LIKE '%@gmail.com'`

(c) '%gmail%' vs '%@gmail.com': Using '%gmail%' is dangerous for auditing because it just looks for that sequence of letters anywhere. It would flag 'john@gmail@yahoo.com' or 'support@gmail.co.uk' as matches. Using '%@gmail.com' is much stricter—it ensures we are specifically filtering for that exact domain at the very end of the string.

Q16 Scenario-Based — Topic: ORDER BY + LIMIT for Rankings

Scenario: You are a sales analyst at an FMCG company. Your VP wants 'Top 10 highest-revenue products this quarter' every Monday. The products table has: product_id, product_name, category, quarterly_revenue, units_sold.

Describe your approach: (a) Which clauses in which order? (b) How to handle ties in quarterly_revenue? (c) How to change the query to return 'Bottom 10 worst-performing products' instead?

Answer: (Write below)

When I get requests for "Top N" or "Bottom N" lists, my brain immediately jumps to sorting and limiting the dataset.

(a) Which clauses in which order? First, I need to sort the data using `ORDER BY quarterly_revenue DESC` (highest to lowest). Then, I restrict

the output to just the first 10 rows using LIMIT 10 at the very end of the query.

(b) Handling ties in quarterly_revenue: If the 10th and 11th products have the exact same revenue, the database will just pick one arbitrarily, which isn't great for reporting. I would add a secondary sort to break the tie logically—perhaps by whichever product moved the most volume. ORDER BY quarterly_revenue DESC, units_sold DESC LIMIT 10;

(c) Bottom 10 worst-performing products: This is an easy fix! I would just change the sort direction from descending to ascending. ORDER BY quarterly_revenue ASC LIMIT 10;

CODING QUESTIONS — SQL (2 Questions)

Q17 Coding — Topic: LIKE + BETWEEN + ORDER BY

Write a SQL query on the customers table (customer_id, full_name, email, city, registration_date) to find customers who: registered between 1 Jan 2023 and 31 Dec 2023 (inclusive), AND have an email ending with '@outlook.com'. Return full_name, email, city, registration_date. Sort by registration_date newest first.

SQL Query: Write your SQL query here *(Write your SQL below)*

```
SELECT
  full_name,
  email,
  city,
  registration_date
FROM
  customers
WHERE
  registration_date BETWEEN '2023-01-01' AND '2023-12-31'
  AND email LIKE '%@outlook.com'
ORDER BY
  registration_date DESC;
```

Q18 Coding — Topic: NOT IN + ORDER BY + LIMIT + Alias

Write a query on orders (order_id, customer_id, product_name, status, order_amount, order_date) to find the 10 most expensive orders NOT in status 'Cancelled' or 'Returned'. Return order_id, product_name, order_amount aliased as 'Order Value', status. Sort by order_amount descending.

SQL Query: Write your SQL query here *(Write your SQL below)*

```
SELECT
order_id,
product_name,
order_amount AS 'Order Value',
status
FROM
orders
WHERE
status NOT IN ('Cancelled', 'Returned')
ORDER BY
order_amount DESC
LIMIT 10;
```

DATABASE SCENARIO-BASED CODING (2 Questions)

Q19 Database Coding — Topic: Multi-Condition WHERE + LIKE + ORDER BY

Database: HR table employees has: emp_id, full_name, department, designation, salary, hire_date, city.

Task: Find all employees with 'Analyst' in their designation, in the 'IT' or 'Data' department, earning more than Rs.70,000. Sort by salary highest first. Return emp_id, full_name, designation, department, salary.

SQL Query: Write your SQL query here (Write your SQL below)

```
SELECT
emp_id,
full_name,
designation,
department,
salary
FROM
employees
WHERE
designation LIKE '%Analyst%'
AND department IN ('IT', 'Data')
AND salary > 70000
ORDER BY
salary DESC;
```

Q20 Database Coding — Topic: BETWEEN + IN + ORDER BY + LIMIT

Database: Pharma table drug_sales has: sale_id, drug_name, region, sale_date, units_sold, revenue, sales_rep.

Task: Find top 5 highest-revenue sales in Q1 2024 (Jan-Mar 2024) from regions 'North', 'South', or 'West'. Show sale_id, drug_name, region, sale_date, revenue aliased as 'Q1 Revenue'. Sort by revenue descending.

SQL Query: Write your SQL query here *(Write your SQL below)*

```
SELECT
  sale_id,
  drug_name,
  region,
  sale_date,
  revenue AS 'Q1 Revenue'
FROM
  drug_sales
WHERE
  sale_date BETWEEN '2024-01-01' AND '2024-03-31'
  AND region IN ('North', 'South', 'West')
ORDER BY
  revenue DESC
LIMIT 5;
```



Learnlytics

.handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.